Λύσεις Δεύτερης Σειράς Ασκήσεων

Λύση Άσκησης 1

1.
Από τα δεδομένα της άσκησης έχουμε:

T(R)=24000 και B(R)=24000/20 = 1200
T(S)=40000 και B(S)=40000/50=800

a. NLJ (Block Nested Loop Join)

Με εξωτερική τη μικρότερη σχέση (S)
cost(NLJ)=B(S) + ceil(B(S)/(M-1)) * B(R) = 800+ceil(800/30)*1200 = 800+27*1200=**33200 I/0.**

Με εξωτερική τη μεγαλύτερη σχέση (R)
cost(NLJ)=B(R) + ceil(B(R)/(M-1)) * B(S) = 1200+ceil(1200/30)*800 = 1200+40*800=**33200 I/O.**

b. SMJ
Επειδή B(R) + B(S) > M^2 (1200+800>31*31) δεν μπορούμε να εφαρμόσουμε την αποδοτική έκδοση του αλγορίθμου. Ισχύει επίσης ότι M<$\sqrt{B(R)}$ (31<$\sqrt{1200}$) και M>$\sqrt{B(S)}$ (31>$\sqrt{800}$) .

Ο αλγόριθμος SMJ θα δημιουργήσει τις παρακάτω ταξινομημένες λίστες για την σχέση S:

- Στο πρώτο πέρασμα θα **διαβάσει** την σχέση S και θα **γράψει** ceil (800/31)= 26 ταξινομημένες λίστες. **Κόστος= 2*B(S)**

Ο αλγόριθμος SMJ θα δημιουργήσει τις παρακάτω ταξινομημένες λίστες για την σχέση R:

- Στο πρώτο πέρασμα θα **διαβάσει** την σχέση R και θα **γράψει** ceil (1200/31)= 39 ταξινομημένες λίστες. **Κόστος= 2*B(R)**
- Στο δεύτερο πέρασμα θα διαβάσει τις 39 ταξινομημένες λίστες και θα τις συγχωνεύσει σε 2 ως εξής: Οι πρώτες 31 λίστες θα συγχωνευτούν σε μία μεγέθους 961 σελίδες και οι υπόλοιπες 8 σε μια δεύτερη λίστα μεγέθους 239 σελίδες. **Κόστος= 2*B(R)**

Στο σημείο αυτό ο αλγόριθμος SMJ έχει δημιουργήσει στον δίσκο **26** ταξινομημένες λίστες για τη σχέση S και **2** ταξινομημένες λίστες για την σχέση R. Η μνήμη χωράει 31 σελίδες οπότε ο αλγόριθμος SMJ προχωρήσει στην συγχώνευση των λιστών και θα δώσει το αποτέλεσμα στην έξοδο. Το κόστος του output δεν προσμετράται. **Κόστος=B(R)+B(S)**

Cost (SMJ)=5*B(R) + 3*B(S) = 5*1200 + 3*800 = 8400.

c. Hash Join
Χρησιμοποιούμε την μικρότερη σχέση S ως build relation και την R ως probe relation. Επειδή B(S) < M^2 (800<31^2) δεν απαιτείται αναδρομική τμηματοποίηση οπότε και το κόστος είναι:

cost(HashJoin)=3*(B(R) + B(S)) = 3 * (1200+800) = 3*2000**=6000 I/0.**

2. Βέλτιστο Κόστος SMJ για M=10

Ο αλγόριθμος SMJ θα δημιουργήσει τις παρακάτω ταξινομημένες λίστες για την σχέση R:

- Στο πρώτο πέρασμα θα **διαβάσει** τη σχέση R και θα **γράψει** 120 ταξινομημένες λίστες μεγέθους **10** σελίδων έκαστη. **Κόστος= 2*B(R).**
- Στο δεύτερο πέρασμα θα **διαβάσει** τις 120 ταξινομημένες λίστες και θα **τις συγχωνεύσει σε** 12 λίστες μεγέθους **100** σελίδων έκαστη. **Κόστος= 2*B(R).**
- Στο τρίτο πέρασμα θα **διαβάσει** τις 12 λίστες και θα τις συγχωνεύσει σε δύο. Μία (1) λίστα μεγέθους 1000 σελίδων και (1) λίστα μεγέθους 200 σελίδων. **Κόστος= 2*B(R).**

Ο αλγόριθμος SMJ θα δημιουργήσει τις παρακάτω ταξινομημένες λίστες για την σχέση S:

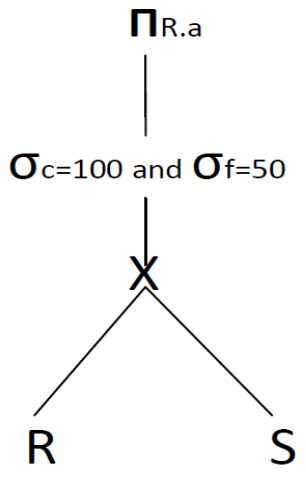
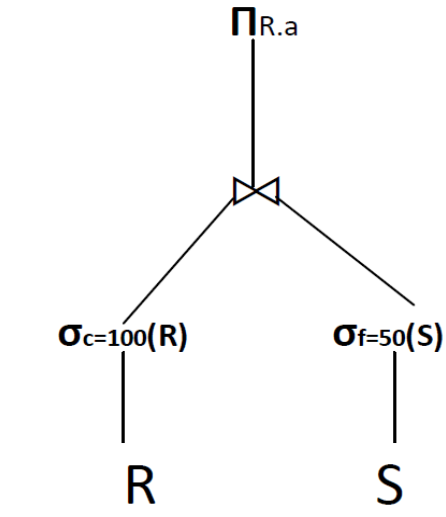
- Στο πρώτο πέρασμα θα **διαβάσει** την σχέση S και θα **γράψει** 80 ταξινομημένες λίστες μεγέθους **10** σελίδων έκαστη. **Κόστος= 2*B(S).**
- Στο δεύτερο πέρασμα θα διαβάσει τις 80 ταξινομημένες λίστες και θα τις συγχωνεύσει σε 8 μεγέθους 100 σελίδες έκαστη. **Κόστος= 2*B(S).**

Στο σημείο αυτό ο αλγόριθμος SMJ έχει δημιουργήσει στον δίσκο **2** ταξινομημένες λίστες για τη σχέση R και **8** ταξινομημένες λίστες για την σχέση S. Η μνήμη χωράει 10 σελίδες οπότε ο αλγόριθμος SMJ προχωρήσει στην συγχώνευση των λιστών και θα δώσει το αποτέλεσμα στην έξοδο. Το κόστος του output δεν προσμετράται. **Κόστος=B(R)+B(S)**

Συνολικό Κόστος = 7*B(R)+5*B(S)= 7*1200+5*800=8400+4000=12400

Λύση Άσκησης 2

1.

Αρχικό Πλάνο	**Τελικό Πλάνο**
	

2.

Από τα δεδομένα έχουμε:

T(R)=30000, B(R)=1500 και V(R,c)=50
T(S)=50000, B(S)=2000 και V(S,f)=100

Σε μία σελίδα χωρούν T(R)/B(R)=30000/1500=20 εγγραφές της R
Σε μία σελίδα χωρούν T(S)/B(S)=50000/2000=25 εγγραφές της S.

Έστω $X=\sigma_{c=100}(R)$
T(X)=T(R)/V(R,c)=30000/50=600 και B(X)=600/20=30 (Non Clustered index on R.c)
Cost(X)=min(1500 με table scan, 600 με την χρήση του ευρετηρίου)=600

Δηλαδή έχουμε: Cost(X)=600, B(X)=30

Έστω $Y=\sigma_{f=50}(S)$
T(Y)=T(S)/V(S,f)=50000/100=500 και B(Y)=500/25=20 (Clustered Index on S.f)
Cost(Y)=min(2000, με table scan, 20 με την χρήση του ευρετηρίου)=20

Δηλαδή έχουμε: Cost(Y)=20, B(Y)=20

a. NLJ (Block Nested Loop Join)

Με εξωτερική σχέση (y)
cost(NLJ)=Cost(Y) + ceil(B(Y)/(M-1)) * Cost(X) = 20+ceil(20/9)*600 = 20 + 3*600=1820 **I/0.**

Με εξωτερική τη σχέση (X)
cost(NLJ)=Cost(X) + ceil(B(X)/(M-1)) * Cost(Y) = 600+ceil(30/9)*20 = 600+ 4*20 = 680 **I/0.**

B. SMJ

Επειδή B(X) + B(Y) < M^2 ($30+20<10^2$) μπορούμε να εφαρμόσουμε την αποδοτική έκδοση του αλγορίθμου.

cost(SMJ)= Cost(X) + Cost(Y) + 2*B(X) + 2*B(Y) =600 + 20 + 2*30 + 2*20 = 720 I/0.

Λύση Άσκησης 3

Ερωτήματα 1,2

Από τα δεδομένα της άσκησης έχουμε:

T(players)=60000
B(players)=600

Μέγεθος Εγγραφής του Πίνακα players=4+34+40+1+1 = 80 bytes
Μέγεθος Σελίδας = 60000/600=100 εγγραφές δηλαδή (100*80)=8000 bytes.

1. Κόστος TableScan= 600 I/O, 60000 εγγραφές στην έξοδο.

2. Κόστος Επιλογής=0 I/O, 21600 εγγραφές στην έξοδο.
Η λειτουργία της επιλογής δεν έχει κόστος I/0. Οι εγγραφές στην έξοδο της επιλογής υπολογίζονται ως εξής:

Βάσει του ιστογράμματος οι εγγραφές που πληρούν το κριτήριο (**age>=20 και age <31)** είναι:

(2/5 * 12000) + (5/5 *24000) +(4/5*18000) = 4800+24000+14400=**43200**

Δεδομένου ότι οι τιμές του πεδίου **rating** κατανέμονται ομοιόμορφα το 50% από τις 43200 εγγραφές αναμένουμε να πληρούν το κριτήριο **rating > 5** δηλαδή **21600.**

3. Κόστος προβολής= 0 I/O, 21600 εγγραφές στην έξοδο.

4. Κόστος I/O=28 I/O, 21600 εγγραφές στην έξοδο.

Το αποτέλεσμα της προβολής είναι 21600 εγγραφές της μορφής **(rating, pid)** δηλαδή μεγέθους 4+1=**5 bytes.** Δεδομένου ότι μία σελίδα έχει χωρητικότητα 8000 bytes, το αποτέλεσμα της προβολής είναι (21600*5)/8000 = 13,5 δηλαδή 1**4 σελίδες.** Το κόστος για την ταξινόμηση των 14 σελίδων είναι 2 * 14 = 28 διότι 14 <=64. (8^2).

5. Κόστος προβολής =0 I/O, 5 εγγραφές στην έξοδο.

Συνολικό Κόστος I/O=600+28=628.

Ερώτημα 3

Αν ο buffer έχει μέγεθος 15 το αποτέλεσμα της προβολής (βήμα 3) χωράει στην μνήμη.

Συνολικό Κόστος = 600 (κόστος table scan).

Λύση Άσκησης 4

1. Επιλογή: Κόστος IO=2 σελίδες, εγγραφές στην έξοδο=10.
Θα χρησιμοποιήσουμε το ευρετήριο συστάδων πάνω στο πεδίο Πόλη. Δεδομένου ότι υπάρχουν 100 διαφορετικές πόλεις και τα δεδομένα κατανέμονται ομοιόμορφα, αναμένουμε να πληρούν το κριτήριο **10** εγγραφές της σχέσης Καταστήματα οι οποίες χωράνε σε **2** σελίδες.

2. INLJ Κόστος IO=10, εγγραφές στην έξοδο=100.
Οι 10 εγγραφές στην έξοδο της επιλογής πρέπει να γίνουν join με την σχέση Πωλήσεις με βάση το γνώρισμα κωδικός. Στο σημείο αυτό μπορούμε να συνδυάσουμε την λειτουργία της ισοσύνδεσης με την επιλογή στο γνώρισμα "Ημερομηνία" ως εξής:

Για κάθε μία από τις 10 εγγραφές που δέχεται ως είσοδο, ο αλγόριθμος INLJ χρησιμοποιεί το σύνθετο ευρετήριο (Κωδικός, ημερομηνία) για να ανακτήσει τις αντίστοιχες εγγραφές της σχέσης ΠΩΛΗΣΕΙΣ (look-up/probe). Βάσει των στατιστικών(κάθε ημέρα γίνονται δέκα πωλήσεις σε κάθε κατάστημα) για κάθε ζεύγος τιμών (Κωδικός, Ημερομηνία) θα ανακτηθούν 10 πλειάδες, δηλαδή μία σελίδα της σχέσης ΠΩΛΗΣΕΙΣ. Επομένως το συνολικό κόστος είναι 10*1=**10 I/O** και στην έξοδο θα έχουμε 10*10=**100** πλειάδες.

3. INLJ Κόστος IO=100, Εγγραφές στην έξοδο=100.
Για κάθε μία από τις 100 πλειάδες που δέχεται ως είσοδο, ο αλγόριθμος INLJ χρησιμοποιεί το ευρετήριο που υπάρχει στο πεδίο barcode της σχέσης ΠΡΟΙΟΝΤΑ για να εντοπίσει και να ανακτήσει τις αντίστοιχες εγγραφές. Δεδομένου ότι το BARCODE είναι πρωτεύον κλειδί του πίνακα ΠΡΟΙΟΝΤΑ, για κάθε BARCODE θα ανακτηθεί μόνο μία εγγραφή, δηλαδή θα διαβαστεί μία σελίδα. Επομένως έχουμε 100*1 = 100 I/O.

Συνολικό Κόστος=2+10+100=112.